

Closing the Sim-to-Real Gap with Less Real Data: Practical Advice and Concrete YOLO11 Changes

Ryan Landay

June 16, 2026

Abstract

These are some ideas for the sim-to-real problem we discussed: a YOLO11 instance-segmentation model trained on Isaac Sim renders (with depth and edge channels available only in simulation) that needs to work on real images, where the real, annotated data is scarce. I have not worked in robotics or sim-to-real directly, so treat this as transferable advice from someone who spends a lot of time getting models to learn well from very little data—my own work is a neural compressor that trains transformers online on datasets as small as 1 MB, which is really a study of data-efficient learning and of injecting structure as a prior rather than as more data. The one idea I would push hardest is a reframing you already gestured at—“bake in the domain adaptation rather than mixing sim and train data.” That instinct is exactly right, and it has a clean, standard formalization: treat the simulator not as data to pile in, but as a *prior whose influence decays as real data accumulates*. The back half of the report turns each suggestion into a concrete change against the Ultralytics YOLO11 API, with code.

1 Framing the problem

The surface problem is sim-to-real; the underlying problem is **learning from a small amount of in-domain (real) data without overfitting, while exploiting an external source of structure (the simulator)**. Three facts shape every suggestion below:

1. **Effective sample size dominates.** With a thin real set, the biggest wins come from how you adapt to that data and how much structure you pre-load, not from a bigger backbone.
2. **The usual scaling intuition misleads.** YOLO defaults are tuned for large datasets. With a few thousand real images you are in a regime where the binding constraint is statistical (parameters fit per real example), so smaller models and stronger priors often win.
3. **Domain knowledge should enter as a prior, not as mixed data.** This is the crux, and the next section makes it precise.

2 Background: the tools in play

For completeness, brief notes on the three systems involved, since the advice below leans on what each one is and is not good at.

Isaac Sim (the simulator). NVIDIA Isaac Sim [10] is a robotics simulator built on the Omniverse platform: PhysX for rigid/soft-body dynamics and RTX path tracing for rendering. Its *Replicator* layer is the relevant part here—it generates labelled synthetic data with pixel-perfect ground truth (instance segmentation masks, depth, surface normals, edges, 2D/3D boxes) and supports domain randomization (textures, lighting, pose). That is why you have depth and edge channels in sim: they are free by-products of a renderer that knows the scene geometry. The sim-to-real gap is then exactly the distribution mismatch between RTX renders and your camera—residual differences in material BRDFs, lighting, sensor noise, and lens/Colour pipeline that the renderer does not reproduce.

YOLO11 (the model). YOLO11 [11, 12] is the 2024 iteration of the YOLO real-time detector family, developed by Ultralytics (the team behind YOLOv5/v8). It is a single-stage, anchor-free convolutional detector: a CSP-style backbone (the new **C3k2** blocks) with an **SPPF** multi-scale pooling stage and a lightweight self-attention block (**C2PSA**, “cross-stage partial with parallel spatial attention”) near the end of the backbone, feeding a multi-scale neck and decoupled detection heads. A single architecture serves five tasks—detection, instance segmentation (**-seg**, your case), pose, oriented boxes, and classification—by swapping the head and loss. It ships in five sizes selected by width/depth multipliers:

Variant	Params	COCO box mAP ₅₀₋₉₅	Rough use
YOLO11n (nano)	2.6 M	39.5	edge / very small data
YOLO11s (small)	9.4 M	47.0	small data
YOLO11m (medium)	20.1 M	51.5	balanced
YOLO11l (large)	25.3 M	53.4	ample data
YOLO11x (extra)	56.9 M	54.7	max accuracy, large data

The variant choice is a capacity knob, and §5 argues it is one of your most important ones: with a thin real set the ladder should be swept from the small end, not defaulted to the largest model that fits. The segmentation variants are `yolo11{n,s,m,l,x}-seg`; the loss is `v8SegmentationLoss` (box + class + DFL + mask terms).

Cosmos (the style-transfer you tried). NVIDIA Cosmos [13] is a family of “world foundation models”; the piece you used, *Cosmos-Transfer*, is a conditional video/image generator built on an adaptive multi-ControlNet: it takes structured controls (segmentation, depth, edge maps—exactly your privileged sim channels) and renders photorealistic output consistent with them, the canonical sim-to-real *augmentation* workflow (turn an Isaac render into a realistic-looking frame while preserving the labels). Its limit, which matches your experience of it “making images more realistic” rather than truly closing the gap, is structural: it improves *pixel* realism but does not align the two distributions in the space your detector actually optimizes (its feature/parameter space). That is the gap the rest of this report attacks directly.

3 The main idea: simulation as a decaying prior, not as mixed data

Your instinct—bake the adaptation in rather than mixing sim and real—has a precise form. The relevant algebra is textbook Bayesian shrinkage [1]; the useful part is the *schedule* it implies.

Treat the simulator-trained model as a prior worth τ_0 *pseudo-examples* of evidence, and the real-data estimate $\hat{\theta}_N$ after N real images as the likelihood. For conjugate pairs the posterior mean is a convex combination,

$$\mathbb{E}[\theta \mid N] = w(N) \theta_{\text{sim}} + (1 - w(N)) \hat{\theta}_N, \quad w(N) = \frac{\tau_0}{\tau_0 + N}, \quad (1)$$

so the correct weight on the sim prior is **hyperbolic in the amount of real data seen**—strong early, fading on a $1/N$ tail—not a constant and not a one-shot fine-tune. Two consequences:

- **Don’t mix sim and real into one training pile.** Mixing implicitly fixes the weight at the data ratio $\frac{|\text{sim}|}{|\text{sim}| + |\text{real}|}$, which is static and, with thousands of sim and few real images, drowns the real signal in the domain you are trying to leave. Equation (1) says the thing to hold fixed is a *pseudo-count* τ_0 , letting the real weight grow on its own.
- **Implement it as a decaying anchor, not a frozen backbone.** Pre-train on sim, then fine-tune on real with a penalty pulling each weight toward its sim value, with strength $\lambda(N) = \lambda_0 \tau_0 / (\tau_0 + N)$ decayed over fine-tuning. Early the model is essentially the sim model; late it is essentially the real model; in between it is the posterior interpolation. The tools are standard—L2-SP [2] or Fisher-weighted EWC [3]; the only twist is making the strength follow $\tau_0 / (\tau_0 + N)$ rather than holding it constant.

This is complementary to your Cosmos/diffusion style transfer, not a replacement. Style transfer attacks the gap in pixel space and plateaus because it is a renderer, not a distribution-aligner; the shrinkage view absorbs the gap in parameter space, with one interpretable knob τ_0 . A style-transferred sim set makes the prior θ_{sim} better; the decaying anchor makes the hand-off to real correct. §10.1 gives the code.

Bayesian shrinkage in one paragraph (with the algebra). Shrinkage is the standard answer to “I have a confident prior belief and a little noisy data—how do I combine them?” Take a scalar parameter θ (read it component-wise for a weight vector). Encode the sim model as a Gaussian prior $\theta \sim \mathcal{N}(\theta_{\text{sim}}, \sigma^2/\tau_0)$ —mean at the sim value, and a precision (inverse variance) of τ_0/σ^2 , i.e. “the prior is worth τ_0 observations of variance σ^2 .” Let the N real images give i.i.d. observations with mean $\hat{\theta}_N$ and per-observation variance σ^2 , so the likelihood has precision N/σ^2 . Multiplying prior by likelihood and completing the square, the posterior is Gaussian with mean equal to the *precision-weighted average*,

$$\mathbb{E}[\theta \mid \text{data}] = \frac{(\tau_0/\sigma^2) \theta_{\text{sim}} + (N/\sigma^2) \hat{\theta}_N}{\tau_0/\sigma^2 + N/\sigma^2} = \frac{\tau_0}{\tau_0 + N} \theta_{\text{sim}} + \frac{N}{\tau_0 + N} \hat{\theta}_N, \quad (2)$$

which is exactly Eq. (1) with $w(N) = \tau_0 / (\tau_0 + N)$. This is the James–Stein/ridge family: pulling a noisy estimate toward a prior mean strictly reduces mean-squared error when data is scarce, the entire reason it helps in the thin-real-data regime. The only modelling choice is τ_0 (how many real images the simulator is “worth”); everything else is forced.

Why $\hat{\theta}_N$ is a likelihood. In Eq. (2) the real data enters *only* through $\hat{\theta}_N$. That is because, for the Gaussian (and any exponential-family) model, the likelihood of the N real images as a function of θ is $\propto \exp(-\frac{N}{2\sigma^2}(\theta - \hat{\theta}_N)^2)$ —a curve peaked at the real-data maximum-likelihood estimate $\hat{\theta}_N$ with width $\sigma/\sqrt{N}$. So “the real data” is summarized by the single sufficient statistic $\hat{\theta}_N$, and the

posterior is literally prior $\times$ likelihood. A deep net has no closed-form $\hat{\theta}_N$, but the fine-tuning loop supplies its operational stand-in: gradient descent on the real images drives the weights toward the real-data optimum (that *is* maximum-likelihood under the segmentation loss), while the anchor penalty supplies $-\log(\text{prior})$. The decaying anchor is thus not a heuristic regularizer—it is the prior term of a MAP estimator whose likelihood is the ordinary real-data loss.

Choosing τ_0 . τ_0 is the crossover sample size: at $N = \tau_0$ the prior and the data carry equal weight ($w = \frac{1}{2}$). The schedule is therefore governed entirely by where τ_0 sits relative to your real budget N_{tot} :

- **Theory / rule of thumb.** Pick τ_0 a modest fraction of N_{tot} so the prior dominates while the real set is genuinely too thin to estimate θ , then fades over the run. $\tau_0 \approx N_{\text{tot}}/4$ keeps the prior at $\sim 20\%$ weight at the end (conservative, stays near sim); $\tau_0 \approx N_{\text{tot}}/16$ spends essentially the whole budget on the $1/N$ tail (aggressive hand-off to real). Outside that band the schedule degenerates— $\tau_0 \rightarrow 0$ ignores the sim model, $\tau_0 \rightarrow \infty$ never lets go of it.
- **Calibrated by experiment in my setting.** The identical $w(N) = \tau_0/(\tau_0 + N)$ governs how fast an engineered prior hands off to the learned model in my compressor (§3, there a position prior rather than sim weights). Using the natural log-schedule it implies, on a $\sim 10^6$ -symbol (“1 MB”) run $\tau_0 = 62,500$ makes the prior fade out right at the end of the run, whereas $\tau_0 = 250,000$ leaves it at one-fifth strength—i.e. $\tau_0 \approx N_{\text{tot}}/16$ used the whole budget and $\tau_0 \approx N_{\text{tot}}/4$ kept the prior too strong, exactly the band above. The practical recipe is the same for you: set τ_0 from your real image count, then sweep it $\times \{0.3, 1, 3\}$ on a *sim* validation split (decisions that need many examples belong on sim, §8).
- **Optimizer-dependent.** A faster learner extracts the likelihood from fewer images, so each image is “worth more” and the prior should fade sooner: empirically (§4) a stronger optimizer wants a proportionally *smaller* τ_0 .

The two anchor implementations: L2-SP and Fisher-EWC. The penalty that realizes the prior comes in an isotropic and an anisotropic form.

- **L2-SP** [2] (“ L^2 to the Starting Point”). Ordinary weight decay penalizes $\|\theta\|^2$, pulling weights toward *zero*; L2-SP instead penalizes $\|\theta - \theta_{\text{sim}}\|^2$, pulling toward the *pretrained* weights. That is precisely $-\log$ of the isotropic Gaussian prior $\mathcal{N}(\theta_{\text{sim}}, \cdot)$ above, so L2-SP *is* the MAP anchor. It is the cheap default and the one in the A1 code.
- **Fisher-weighted EWC** [3] (Elastic Weight Consolidation). Replaces the isotropic ball by a per-parameter Fisher-weighted one, $\sum_i F_i (\theta_i - \theta_{\text{sim},i})^2$, where F_i is the diagonal Fisher information of the sim task at θ_{sim} . This is the Laplace (Gaussian) approximation to the *sim-task posterior*: directions the sim solution constrained tightly (large F_i) are held nearly fixed, while directions it left loose move freely. Originally proposed against catastrophic forgetting, here it makes the anchor anisotropic in exactly the right basis—you keep the structure the simulator actually pinned down and let the rest re-fit to real data. Use it when isotropic L2-SP either over-constrains (too rigid) or under-constrains (lets important sim structure drift).

Evidence from the compressor that the *shape* matters. Two results from my own runs support the decaying-prior recipe over the static alternatives (these are language-compression numbers in bits/byte, offered as directional support for the schedule shape, not as vision benchmarks):

- **The $\tau_0/(\tau_0 + N)$ hand-off shape wins over flat and over a too-tight floor.** Applying the conjugate decay-to-floor schedule $\eta(N) = \eta_\infty + (\eta_0 - \eta_\infty)\tau/(\tau + N)$ to the learning rate on a 24-layer model at the 10 MB scale, the decayed schedule with a $0.625\times$ floor reached 1.5370 bits/byte versus 1.5466 for the flat (no-decay) baseline, and beat a tighter $0.25\times$ floor (1.5510). So among hand-off shapes, *decay along $\tau/(\tau + N)$ toward a non-zero floor* was best—neither “hold the prior fixed” (flat) nor “drop it almost entirely” (tight floor). The exact floor matters; the analogue for you is that L2-SP’s $\lambda(N)$ should decay toward a small non-zero value, not to zero, if the sim prior carries any permanently useful structure. (Caveat: that specific cell was confounded by an unrelated low-precision flag; I cite it as directional.)
- **Capacity tracks effective data** (used in §5): the arg-min model depth rose from 22 layers at the 10^6 -symbol scale to 24 at 10^7 , with 26 layers measurably worse at the smaller scale—a clean instance of “more in-domain data supports more capacity,” whose dual (“a stronger prior wants a smaller model”) is the coupling in §5.

4 Pre-train, reset the optimizer, then fine-tune

Standard pre-train/fine-tune, with one discipline that is easy to skip:

1. Pre-train YOLO11 on the abundant Isaac Sim renders to convergence.
2. **Reset the optimizer state** (Adam/AdamW moments and the LR schedule) before touching real data—keep the *weights*, drop the *optimizer history*. Carrying stale moments injects sim-domain curvature into the first, disproportionately influential real updates.
3. Fine-tune on real under the decaying anchor of §3 with a fresh short LR warmup/cosine schedule.

In Ultralytics you get the optimizer reset for *free* by running two separate `model.train()` calls (sim, then real loading the sim checkpoint): each `train()` builds a fresh optimizer. The thing to avoid is one `train()` over a mixed dataset.

Aside: would Muon help, and does it change this advice? You may have seen Muon [14], the optimizer that has been winning language-model speed-runs. It is worth knowing where it does and does not transfer, because it interacts with both the “learn from less data” goal and the reset discipline. What Muon does: for each 2D weight *matrix* it replaces the (momentum-averaged) gradient by its orthogonalization—the polar factor UV^T computed with a few Newton–Schulz iterations—which equalizes the update magnitude across all singular directions, so even rarely-excited directions take a full-size step instead of being starved. Three things follow for YOLO:

- **It is not a drop-in for a conv net.** Muon’s orthogonalization is defined for 2D matrices; YOLO11 is mostly 4D conv tensors plus BatchNorm and a few linear/attention layers (the C2PSA block). Out of the box Muon would only touch the 2D layers and leave every conv on Adam—i.e. almost the whole network. To put convs on Muon you reshape each filter $[c_{\text{out}}, c_{\text{in}}, k, k] \rightarrow [c_{\text{out}}, c_{\text{in}}k^2]$ and orthogonalize that, which is a real change to validate, not a free switch.
- **It plausibly does learn from fewer images—on the matmul-heavy parts.** In my runs Muon’s content path reaches a given fit in fewer symbols than Adam (a measured “Muon learns a factor κ faster” crossover). In the shrinkage language of §3 that means each real example is worth κ times more evidence, so with Muon you would use a *weaker, faster-fading* prior ($\tau_0^{\text{Muon}} \approx \tau_0^{\text{Adam}}/\kappa$) and could afford a slightly smaller model. The faster effective convergence

per real image is exactly the property you want for scarce real data—but the gain is largest on attention/linear layers, so expect a muted version on a conv-dominated detector.

- **Yes, it still needs the reset.** Muon is not stateless: it carries a momentum buffer (and the practical “NorMuon” variant adds an Adam-style per-entry second-moment normalizer). Those buffers encode sim-domain scale and curvature just like Adam’s moments, so the same keep-weights/drop-state discipline applies—two separate `train()` calls. One mild nuance: because the Newton–Schulz step re-orthogonalizes from the *current* gradient every step, Muon’s direction depends less on stale momentum than Adam’s does, so a carried-over Muon state is somewhat less damaging than carried-over Adam moments. Resetting is still the safe default.

Net: Muon is worth a back-burner experiment (reshape convs, weaken τ_0), but it is a bigger change than the rest of this report and the reset advice is unchanged either way.

5 Match capacity to the real data (bias–variance)

In the scarce-data regime the binding constraint is parameters fit per real example. That argues for matching model capacity to the real set, and for letting the prior carry structure while data is thin:

- With a few thousand real images, **prefer a smaller YOLO11 variant (n/s/m) over l/x**, or lean harder on the sim anchor (§3); a large backbone fine-tuned on a thin set overfits the transient. Sweep the variant ladder instead of defaulting to the biggest model that fits.
- **Capacity and prior strength are coupled:** a stronger anchor should *lower* the best model size and vice versa—tune them together.
- Useful falsifier: if a *smaller* model keeps winning as you add real data, the bottleneck is optimization, not capacity, and effort should move to the schedule/augmentation, not the backbone.

The bias–variance algebra, and why capacity grows with data. Write the expected real-domain risk of a fitted model with p effective trainable parameters, on N real images, in the textbook decomposition

$$\mathcal{R}(p) \approx \underbrace{B^2(p)}_{\text{approximation bias}} + \underbrace{c \frac{p}{N_{\text{eff}}}}_{\text{estimation variance}} + \sigma_{\text{noise}}^2, \quad (3)$$

where $B^2(p)$ falls as the model gets more flexible (more $p \Rightarrow$ it can represent the true segmentation function more closely) and the variance term rises linearly in p/N_{eff} (each parameter must be pinned down from finite data). Minimizing over p balances a *decreasing* bias against an *increasing* variance, giving an interior optimum p^* ; because the variance slope is c/N_{eff} , growing N_{eff} flattens it and pushes p^* *up*. That is the measured capacity drift cited in §3 (arg-min depth 22 $\rightarrow$ 24 as data went 10 $\times$), and it is the same two-term shape that governs the optimal learning rate in my notes,

$$\mathcal{E}(\eta) \approx \underbrace{c_1 \eta \sigma^2}_{\text{noise rides the step}} + \underbrace{c_2 \delta^2 / \eta}_{\text{lag behind the drift}}, \quad \eta^* = \sqrt{c_2 / c_1} \delta / \sigma, \quad (4)$$

an explicit “one term grows, one term shrinks, optimum in the middle” tradeoff (here for a step size rather than a parameter count) that I verified empirically as the decay-to-floor LR result of §3.

Why a stronger anchor lowers the best model size (and vice versa). The anchor enters Eq. (3) in two ways. First, a Gaussian prior of strength τ_0 acts like extra effective data, $N_{\text{eff}} \rightarrow N + \tau_0$, which on its own would *raise* p^* . But—and this is the dominant effect—the prior also *supplies structure the model would otherwise have to learn with its own parameters*. A strong anchor constrains the fit to a tight neighborhood of θ_{sim} (in the Fisher/EWC view it nearly freezes the high-information directions), so those directions are no longer free degrees of freedom that the real data must estimate; the model’s representational content is partly furnished by the prior rather than by its parameters. Adding capacity under a strong anchor therefore buys little new bias reduction (new parameters are pulled back toward their sim values) while still paying variance on the slack directions the anchor does *not* constrain—which is exactly where overfitting lives. The optimum free-capacity p^* thus *falls* as τ_0 rises. This is the parameter-count instance of a substitution I see in the compressor’s position prior: as the learned path acquires structure the fixed prior is shed, and conversely a stronger fixed prior lets the learned path be smaller. Practically: **tune model size and anchor strength together**—a strong sim anchor pairs with YOLO11n/s; weaken the anchor before you reach for $1/x$.

6 Small batches, more updates

When data is scarce, the number of gradient updates the model takes while consuming it matters: smaller batches give more, finer updates over the same examples, so the model is “further trained” by the time it sees later data. This is the classic small-batch SGD advantage, and it bites hardest exactly when you are far from convergence—i.e. small datasets. (In my compressor, where the model trains as it codes, this online-adaptation count turned out to be the single largest lever on compression, which is what makes me flag it here.) Practical reading: when fine-tuning on the real set, **favour smaller batches / more steps** over large batches and few steps, within the optimizer’s noise tolerance, and remember that gradient accumulation (which enlarges the effective batch for throughput) is *not* free here—it trades adaptation depth for speed. Find the knee on a held-out real split.

7 Curriculum: easy real images first

A light curriculum helps small sets: score real images by a cheap proxy (the sim-pretrained model’s own loss/confidence, or scene clutter / instance count), warm up on the easy, on-distribution images, then introduce the hard, off-sim ones. This is standard curriculum learning [4]; difficulty alone tends to cluster trivially, so combine it with a content signal (e.g. embedding distance to the sim distribution).

8 Methodology: validate on sim, spend real data on the final fit

Two process points, given your asymmetric budget (abundant sim, scarce real):

- **Use sim to make the decisions that need many examples** (architecture, augmentation policy, hyperparameters, curriculum) and reserve real images for the actual adaptation.
- **Localize the gap before attacking it.** You said you are not sure where the gap comes from—that is the first thing to fix. Train on sim, evaluate per-class / per-condition on the real validation set, and see where error concentrates (small objects? particular textures? lighting? occlusion?).

An RKHS/MMD distance [5] is a fine global gap statistic, but a per-stratum error breakdown tells you *which* knob to pull.

What RKHS/MMD is (since you used it). A Reproducing Kernel Hilbert Space (RKHS) is a function space attached to a kernel $k(\cdot, \cdot)$ in which a whole distribution P can be summarized by a single point—its *kernel mean embedding* $\mu_P = \mathbb{E}_{x \sim P}[k(x, \cdot)]$. For a characteristic kernel (e.g. Gaussian RBF) this embedding is lossless: μ_P encodes all moments of P . The *Maximum Mean Discrepancy* is then just the RKHS distance between the two embeddings,

$$\text{MMD}^2(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}^2 = \mathbb{E} k(x, x') + \mathbb{E} k(y, y') - 2 \mathbb{E} k(x, y),$$

with $x, x' \sim P$ (sim) and $y, y' \sim Q$ (real); the right-hand side is estimated directly from samples, no density estimation required [5]. So your RKHS experiment *is* an MMD between sim and real (typically computed on backbone features rather than raw pixels). Two cautions: it is a single global scalar, so it tells you the domains differ but not *which* stratum is to blame—hence the per-condition breakdown above; and it is most useful not as a diagnostic but as a differentiable *loss* (penalize feature-MMD during training), of which the adversarial form (DANN, §10/A5) is the higher-capacity cousin: a domain classifier learns the divergence that MMD fixes by choice of kernel.

9 Using the privileged sim channels (depth, edges)

You have depth and edges in sim but not at deployment. Information available only at training time is *privileged information*; the right way to use it is not as an input (you cannot at test time) but as *auxiliary training signal*: add auxiliary heads predicting depth/edges during the sim phase, or distil from a depth/edge-conditioned teacher into the RGB-only student [6]. This makes the sim phase build a richer prior θ_{sim} , which the decaying anchor then carries into real training.

10 Speculation: architecture changes, with code

The suggestions above mostly change *training*. Here are structural changes too. I have not validated any of these on a vision model—they are speculative and each is a cheap A/B against your sim-only baseline. The modules themselves are standard domain-adaptation tools; briefly, so the rest of the section has names to hang on:

- **Domain-adversarial branches.** A side “domain classifier” reads a backbone feature and tries to tell sim from real; a gradient-reversal layer flips the sign of its gradient on the way back, so the backbone is trained to *defeat* it and produce features whose sim/real distributions are indistinguishable. This is DANN [9] (A5)—an adversarial, learned version of minimizing the MMD of §8.
- **Domain-specific normalization.** Keep *separate* normalization statistics per domain (separate BatchNorm running mean/var, or instance norm) while sharing all convolution weights, so each domain is standardized to its own statistics. Cheap and parameter-light; AdaBN [8] (A4) is the train-on-sim/re-estimate-on-real special case.
- **Adapters.** Small trainable modules (bottleneck MLPs, or low-rank convolutions) inserted into an otherwise *frozen* backbone; only the adapters move on real data, capping the number of parameters the scarce real set has to fit [16, 17, 7] (A3).

- **FiLM (feature-wise linear modulation).** Condition the network by applying a per-channel affine map $\gamma \odot h + \beta$ to feature maps h , where (γ, β) are produced by a tiny network from a conditioning code [19]—here a domain indicator or a privileged-channel embedding. A lightweight way to inject a domain signal on a *separate pathway* without disturbing the backbone weights.

What carries over from data-efficient training is the *framing*: isolate the domain-specific part so few parameters re-fit on real data, inject domain knowledge on a separate pathway rather than contaminating the main features, and exploit structure you already know instead of learning it.

All snippets target the Ultralytics API. The pattern is: subclass `SegmentationTrainer` and pass it via `model.train(..., trainer=MyTrainer)`; override `get_model` to swap the loss/model, `build_optimizer` to set parameter groups, and register callbacks with `add_callback`.

10.1 (A1) Decaying sim-anchor as a custom loss + callback

This is the concrete form of §3: wrap the segmentation loss to add $\lambda(t) \sum_i (\theta_i - \theta_i^{\text{sim}})^2$, snapshot the sim weights when the model loads them, and decay λ on the $\tau_0/(\tau_0 + \text{step})$ tail.

```
import torch
from ultralytics import YOLO
from ultralytics.models.yolo.segment import SegmentationTrainer
from ultralytics.nn.tasks import SegmentationModel
from ultralytics.utils import RANK
from ultralytics.utils.loss import v8SegmentationLoss

class AnchoredSegLoss(v8SegmentationLoss):
    """Segmentation loss + decaying L2 anchor to the sim-pretrained weights."""

    def __init__(self, model, tau0=2000.0, lam0=1.0):
        super().__init__(model)
        self.model = model
        self.tau0, self.lam0 = tau0, lam0
        self.step = 0 # advanced by the trainer callback
        # snapshot sim weights as frozen reference (theta_sim)
        self.ref = {n: p.detach().clone()
                     for n, p in model.named_parameters() if p.requires_grad}

    def lam(self):
        return self.lam0 * self.tau0 / (self.tau0 + self.step) # w(N) tail

    def __call__(self, preds, batch):
        loss, items = super().__call__(preds, batch) # base seg loss (scaled)
        pen = sum(((p - self.ref[n]) ** 2).sum()
                  for n, p in self.model.named_parameters()
                  if n in self.ref and p.requires_grad)
        return loss + self.lam() * pen, items

class AnchoredSegModel(SegmentationModel):
    def init_criterion(self):
        return AnchoredSegLoss(self, tau0=2000.0, lam0=1.0)

def _advance_anchor_step(trainer):
    crit = trainer.model.criterion
    if isinstance(crit, AnchoredSegLoss):
        crit.step += 1 # N grows -> lambda decays on tau0/(tau0+N)
```

```

class AnchoredTrainer(SegmentationTrainer):
    def __init__(self, *a, **k):
        super().__init__(*a, **k)
        self.add_callback("on_train_batch_end", _advance_anchor_step)

    def get_model(self, cfg=None, weights=None, verbose=True):
        model = AnchoredSegModel(cfg, nc=self.data["nc"], ch=self.data["channels"],
                                verbose=verbose and RANK == -1)

        if weights:
            model.load(weights)      # load sim weights BEFORE the snapshot below
            model.init_criterion()    # snapshot theta_sim from the loaded weights
        return model

# Phase 1: pretrain on sim (fresh optimizer). Phase 2: anchored fine-tune on real.
YOLO("yolo11s-seg.yaml").train(data="sim.yaml", epochs=100)      # -> sim.pt
YOLO("sim.pt").train(data="real.yaml", epochs=50, trainer=AnchoredTrainer)

```

Where the added term comes from. Minimizing $L_{\text{seg}}(\theta) + \lambda \sum_i (\theta_i - \theta_i^{\text{sim}})^2$ is *maximum-a-posteriori* estimation under the Gaussian prior $\theta \sim \mathcal{N}(\theta_{\text{sim}}, \frac{1}{2\lambda}I)$: the objective is $-\log p(\text{data} | \theta) - \log p(\theta)$ up to a constant, with the segmentation loss playing the role of the negative log-likelihood (§3) and the quadratic playing $-\log$ prior. So the term is not an ad-hoc regularizer—it is exactly the prior of the shrinkage model, which is why decaying it as $\lambda(N) = \lambda_0 \tau_0 / (\tau_0 + N)$ reproduces the posterior weight $w(N)$ of Eq. (1). The Fisher/EWC variant replaces the isotropic $\mathbf{I}$ by $F_i(\theta_i - \theta_i^{\text{sim}})^2$, i.e. a Gaussian prior with covariance $\text{diag}(1/F_i)$ —the Laplace approximation to the sim posterior.

Choosing λ_0 and τ_0 . They control different things. τ_0 is the *crossover* (§3): set it from your real image count— $\tau_0 \approx N_{\text{tot}}/4$ to stay conservative, $N_{\text{tot}}/16$ to hand off aggressively—then sweep $\times \{0.3, 1, 3\}$ on a sim-validation split. λ_0 fixes the prior’s *units*, which depend on the loss scale and parameter count, so calibrate it once empirically: on a single real batch at $N = 0$, print $\lambda_0 \|\theta - \theta^{\text{sim}}\|^2$ and the seg loss, and pick λ_0 so the anchor term is $\sim 0.1\text{--}1 \times$ the seg loss (neither ignored nor dominating). Because `v8SegmentationLoss` is already batch-scaled, fixing that ratio once transfers across batch sizes. Step `tau0` per *image* rather than per batch if you want N to count examples exactly.

Evidence this works. The two components are individually well-supported: L2-SP [2] and EWC [3] both report consistent gains for transfer onto small/dissimilar targets, which is your regime. What I can add from my own work is that the *decaying* schedule beats the static alternatives: applying the same $\tau_0/(\tau_0 + N)$ hand-off shape (to the learning rate, the closest lever I have) beat both a frozen schedule and a too-fast drop at the 10^7 scale (§3), and the prior-strength anneal that uses this exact $w(N)$ is part of my best configuration—freezing it (constant λ) is worse. The directional claim I would stand behind: prefer the decayed $\lambda(N)$ over a constant penalty, and floor it at a small non-zero value rather than at zero.

10.2 (A2) Put the measured object-size / class prior into the head

YOLO11’s detect head initializes its class bias to a fixed object-frequency prior (in `head.py`, `bias_init` sets the cls bias to `math.log(5 / nc / (640 / stride)) ** 2`—“5 objects, nc classes, 640 img”). You measured your objects and render them from a 3D replicator, so you know the real per-class frequency and scale distribution: initialize from *that* instead of the generic prior, so the head starts

at the right operating point and the scarce real data only has to refine it. A callback after model setup:

```
import math, torch
from ultralytics.nn.modules.head import Detect

# measured expected count of each class per image (from your sim/measurements)
EXP_COUNT = torch.tensor([0.8, 0.2, 1.5]) # len == nc

def set_measured_cls_prior(trainer):
    det = next(m for m in trainer.model.modules() if isinstance(m, Detect))
    cls_heads = det.cls_head if hasattr(det, "cls_head") else det.cv3 # version-robust
    for conv, s in zip(cls_heads, det.stride):
        n_loc = (640 / s) ** 2 # locations at this scale
        conv[-1].bias.data[:det.nc] = torch.log(EXP_COUNT / n_loc + 1e-9)

# register so it runs once after the model/strides are ready, before training
trainer.add_callback("on_pretrainRoutine_end", set_measured_cls_prior)
```

The scale half of the prior goes into the data config rather than the head: set the augmentation `scale/imgsz` range to match your measured object-size distribution so the model trains on the scales it will actually see, rather than COCO’s defaults.

How the class-frequency prior affects training. Dense detectors are extremely class-imbalanced: almost every spatial location is background, so at random init the classification loss is dominated by pushing a huge number of locations from $p \approx 0.5$ down toward background. Initializing the cls bias to a low foreground prior $\log(\frac{\pi}{1-\pi})$ starts every location already predicting “probably background,” so the first gradients are spent on the rare *foreground* corrections that actually matter, not on the bulk re-centering—it stabilizes the early loss and speeds convergence. This is precisely the focal-loss “prior π ” trick from RetinaNet [15], and YOLO ships a generic version of it in `bias_init`. Replacing the generic `5-objects-per-640px` prior with *your measured* per-class rates means each class’s head starts at its true base rate, so the scarce real data only refines the decision boundary instead of first having to discover how common each class is.

Evidence this works. RetinaNet’s ablation is direct: without the bias-prior initialization, training of a dense detector is unstable and degrades sharply (their reported drop is large; they call it necessary for getting dense detection to train at all) [15]. That YOLO carries the same prior in its own `bias_init` is corroborating evidence the maintainers found it necessary in practice. The measured-frequency refinement is a strict specialization of a technique already known to help.

10.3 (A3) Freeze the backbone, adapt a small module (cap real-data parameters)

The architectural form of “few parameters re-fit on real data” (§5): freeze the sim-trained backbone and train only the head plus the BatchNorm affine parameters (a cheap, built-in “adapter”), with a higher LR on the trainable part. This is a `build_optimizer` override plus the built-in `freeze`:

```
from ultralytics.models.yolo.segment import SegmentationTrainer
from ultralytics.utils.torch_utils import unwrap_model
import torch

class AdapterTrainer(SegmentationTrainer):
    def build_optimizer(self, model, name="auto", lr=0.001, momentum=0.9,
                       decay=1e-5, iterations=1e5):
        trainable = [p for p in unwrap_model(model).parameters() if p.requires_grad]
        # AdamW; head+BN-affine left trainable by 'freeze', backbone convs frozen
```

```

return torch.optim.AdamW(trainable, lr=lr, weight_decay=decay)

# freeze=11 freezes the YOLO11 backbone (layers 0-10); head (11+) stays trainable
YOLO("sim.pt").train(data="real.yaml", epochs=50, freeze=11, trainer=AdapterTrainer)

```

If you want true low-rank adapters (LoRA-style [7]), insert a small parallel 1×1 conv beside each frozen backbone conv and add only those to the optimizer; the freeze-plus-BN version above is the 10-line approximation and is usually enough.

Why “usually enough” is a defensible claim. Parameter-efficient transfer is one of the better-validated results in the area. Housby et al. [16] match full fine-tuning to within $\sim 0.4\%$ on GLUE while training only $\sim 3\%$ of the parameters; Rebuffi et al. [17] adapt a single frozen visual backbone to many domains with small residual adapters; and LoRA [7] reaches full-fine-tune quality with a tiny low-rank update. Most relevant to sim-to-real specifically: Kumar et al. [18] show that *full* fine-tuning can distort pretrained features and *underperform* on out-of-distribution targets, exactly where freezing/adapters help—so with a thin, shifted real set, capping the adapted parameters is not just cheaper but often *better*. That is the basis for “usually enough.”

10.4 (A4) Domain-specific BatchNorm (AdaBN)

A large part of a sim-real gap is low-order channel statistics (brightness, contrast, texture energy), which live entirely in BatchNorm running stats. AdaBN [8] re-estimates those on real data without touching the convolutions. The cheapest version: after sim pre-training, reset BN running stats and let a few real-data forward passes repopulate them.

What AdaBN is doing. Each BatchNorm layer stores per-channel running mean/variance and standardizes its input by them: $\hat{h} = (h - \mu_c) / \sqrt{v_c}$. Those statistics are domain-specific—the sim renderer and the real camera differ most in exactly such low-order channel moments (overall brightness, contrast, texture energy). AdaBN’s observation is that simply *recomputing* μ_c, v_c on target-domain data (no labels, no backprop) re-centers and re-scales every feature map into the real domain’s regime before the shared convolutions see it, removing a large part of the shift for essentially zero cost. The affine γ, β and the conv weights are left to the rest of the pipeline (or to a short fine-tune).

```

import torch.nn as nn

def adabn_reset(trainer):
    if trainer.epoch == 0:
        for m in trainer.model.modules():
            if isinstance(m, nn.BatchNorm2d):
                m.reset_running_stats()      # forget sim-domain BN statistics
                m.momentum = None           # use cumulative moving average
# stats are then re-accumulated from the real images during the first epoch(s)
trainer.add_callback("on_train_epoch_start", adabn_reset)

```

Evidence this works. AdaBN [8] reported state-of-the-art unsupervised domain adaptation on the standard Office and Caltech-256 benchmarks of its day with a parameter-free, backprop-free statistics swap, and the trick has been widely reproduced and folded into later DA methods. It is the highest reward-to-effort item here precisely because it costs nothing to try—one callback—and targets the part of the gap (low-order channel statistics) that a renderer-vs-camera mismatch most reliably produces.

10.5 (A5) Make domain-invariance an architectural objective (DANN)

This turns your existing MMD work into a training signal. Add a small domain classifier that reads a backbone feature through a gradient-reversal layer [9], so the backbone is pushed to produce features that cannot distinguish sim from real. Higher-variance, higher-upside than A1–A4; sketch:

```
import torch, torch.nn as nn

class GradReverse(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lam):
        ctx.lam = lam; return x.view_as(x)
    @staticmethod
    def backward(ctx, g):
        return -ctx.lam * g, None          # reverse + scale the gradient

class DomainHead(nn.Module):
    def __init__(self, c):                  # c = channels of the tapped feature
        super().__init__()
        self.net = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                   nn.Linear(c, 128), nn.ReLU(), nn.Linear(128, 1))
    def forward(self, feat, lam):
        return self.net(GradReverse.apply(feat, lam)).squeeze(1)
# Tap a mid/late backbone feature (e.g. model.model[10]) via a forward hook,
# pass it through DomainHead, and add BCE(domain_logit, is_real) to the loss in
# init_criterion. Mix sim+real *batches* here (this is the one place mixing helps,
# because the domain label is the supervision). Ramp 'lam' from 0 -> 1 over training.
```

What DANN is, precisely. A domain-adversarial neural network [9] attaches a domain classifier D to a backbone feature and trains it to predict sim-vs-real, while the gradient-reversal layer multiplies D 's gradient by $-\lambda$ as it flows back into the backbone. The backbone therefore *descends* the task loss but *ascends* the domain loss—it is pushed to make sim and real features indistinguishable to the best available classifier, even as the task head keeps them discriminative for segmentation. Formally this is a saddle-point / adversarial estimate of the domain divergence (the $\mathcal{H}$ -divergence), which is the learned-kernel counterpart of the fixed-kernel MMD you already compute (§8)—so A5 is your RKHS experiment turned from a measurement into an objective. Ramping λ from 0 avoids the domain branch destabilizing early training.

Evidence this works. DANN [9] reports large unsupervised-domain-adaptation gains on standard shifts (e.g. MNIST→MNIST-M, SVHN→MNIST, and the Office benchmark), and gradient reversal has become a standard baseline that many later sim-to-real methods build on. It is the highest-variance item here—an adversarial term needs tuning (λ schedule, where to tap)—but it is also the principled home for the distribution-alignment instinct you started from (“bake in the domain adaptation”), realized in feature space rather than pixel space.

Priority. Cheapest-first: **A1** (decaying anchor) and **A3/A4** (freeze + BN adaptation) are a few lines on an existing loop and directly implement “few parameters re-fit on real data.” **A2** is nearly free and uniquely available to you since the object sizes are measured. **A5** is the bigger bet and is the natural home for your MMD/RKHS work.

11 Summary

Suggestion	What to do	Type
Sim as a decaying prior (§3, A1)	Anchor real fine-tuning to the sim weights with strength $\tau_0/(\tau_0 + N)$; don't mix sim+real into one pile	training + loss
Pre-train, reset optimizer, fine-tune (§4)	Two separate <code>train()</code> calls (sim then real) gives the moment reset free	procedure
Match capacity to real data (§5)	Prefer smaller YOLO11 (n/s/m); couple model size to anchor strength	model size
Small batch, more updates (§6)	Smaller batches / more steps on the real set; accumulation isn't free	schedule
Curriculum (§6)	Easy/on-distribution real images first	data order
Localize the gap first (§7)	Per-stratum real error breakdown before more transfer	methodology
Privileged channels (§9)	Distil depth/edge into auxiliary sim-phase heads; RGB-only at test	aux loss
Measured prior into head (A2)	Init cls bias from measured frequencies; scale aug to measured sizes	head init
Freeze + adapter (A3)	Train head + BN affine only, frozen backbone	architecture
AdaBN (A4)	Reset/re-estimate BN stats on real data	architecture
DANN / MMD branch (A5)	Gradient-reversal domain head for invariant features	architecture

If you try one thing first. Replace data-mixing with the decaying sim-anchor of §3/A1: pre-train on sim, reset the optimizer, and fine-tune YOLO11 on the real set with anchor strength $\tau_0/(\tau_0 + N)$. It is a small change to an existing loop, it implements the “bake it in, don't mix” idea directly, and it gives one interpretable knob (τ_0) instead of an opaque data ratio.

References

- [1] P. Diaconis and D. Ylvisaker. Conjugate priors for exponential families. *Annals of Statistics*, 1979.
- [2] L. Xuhong, Y. Grandvalet, and F. Davoine. Explicit inductive bias for transfer learning with convolutional networks (L2-SP). *ICML*, 2018.
- [3] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 2017.
- [4] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. *ICML*, 2009.
- [5] A. Gretton et al. A kernel two-sample test (MMD). *JMLR*, 2012.
- [6] D. Lopez-Paz, L. Bottou, B. Schölkopf, and V. Vapnik. Unifying distillation and privileged information. *ICLR*, 2016.
- [7] E. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

- [8] Y. Li, N. Wang, J. Shi, J. Liu, and X. Hou. Revisiting batch normalization for practical domain adaptation (AdaBN). *ICLR Workshop*, 2017.
- [9] Y. Ganin et al. Domain-adversarial training of neural networks. *JMLR*, 2016.
- [10] NVIDIA. Isaac Sim: robotics simulation and synthetic data generation on Omniverse. NVIDIA Developer documentation, 2024. <https://developer.nvidia.com/isaac/sim>.
- [11] R. Khanam and M. Hussain. YOLOv11: an overview of the key architectural enhancements. *arXiv:2410.17725*, 2024.
- [12] G. Jocher and J. Qiu (Ultralytics). Ultralytics YOLO11. Software and documentation, 2024. <https://docs.ultralytics.com/models/yolo11>.
- [13] NVIDIA. Cosmos-Transfer1: conditional world generation with adaptive multimodal control. *arXiv:2503.14492*, 2025.
- [14] K. Jordan et al. Muon: an optimizer for the hidden layers of neural networks. Technical report, 2024. <https://kellerjordan.github.io/posts/muon/>.
- [15] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection (RetinaNet). *ICCV*, 2017.
- [16] N. Houlsby et al. Parameter-efficient transfer learning for NLP. *ICML*, 2019.
- [17] S.-A. Rebuffi, H. Bilen, and A. Vedaldi. Learning multiple visual domains with residual adapters. *NeurIPS*, 2017.
- [18] A. Kumar et al. Fine-tuning can distort pretrained features and underperform out-of-distribution. *ICLR*, 2022.
- [19] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: visual reasoning with a general conditioning layer. *AAAI*, 2018.